Course Code: CSE2261

Course Title: DBMS Lab

Labsheet 9

Create the database using the given schema. (Student Library)

Exp: Demonstrate the schema's relationships and data integrity by creating and linking tables as per the specified requirements.

Database Creation (Schema + Relationships)
Create Database

```
CREATE DATABASE StudentLibrary;
USE StudentLibrary;
```

Tables Creation

1. Students Table

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    email VARCHAR(100) UNIQUE
);
```

2. Books Table

```
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(150),
    author VARCHAR(100),
    total_copies INT CHECK (total_copies >= 0),
    available_copies INT CHECK (available_copies >= 0)
);
```

3. Librarians Table

```
CREATE TABLE Librarians (
    librarian_id INT PRIMARY KEY,
```

```
    name VARCHAR(100),
    contact VARCHAR(15)
);
```

4. Borrow Table (Relationship Table)

Connects **Students ↔ Books ↔ Librarians**

```
CREATE TABLE Borrow (
    borrow_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    book_id INT,
    librarian_id INT,
    issue_date DATE,
    return_date DATE,

    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (librarian_id) REFERENCES Librarians(librarian_id)
);
```

Insert Sample Data

```
INSERT INTO Students VALUES
(1, 'Ravi', 'CSE', 'ravi@gmail.com'),
(2, 'Anu', 'ECE', 'anu@gmail.com');

INSERT INTO Books VALUES
(101, 'DBMS', 'Korth', 5, 5),
(102, 'AI Basics', 'Russell', 3, 3);

INSERT INTO Librarians VALUES
(1, 'Meena', '9876543210');
```

SQL Queries

Query 1: Students who borrowed books but NOT returned

```
SELECT s.name, b.title
FROM Students s
JOIN Borrow br ON s.student_id = br.student_id
```

```
JOIN Books b ON br.book_id = b.book_id
WHERE br.return_date IS NULL;
```

- `JOIN` connects tables
- `return_date IS NULL` → book not returned

Query 2: Books never borrowed

```
SELECT title
FROM Books
WHERE book_id NOT IN (SELECT book_id FROM Borrow);
```

- Subquery finds borrowed books
- Outer query finds missing ones

Query 3: Count books borrowed per student

```
SELECT s.name, COUNT(br.book_id) AS total_books
FROM Students s
LEFT JOIN Borrow br ON s.student_id = br.student_id
GROUP BY s.name;
```

- `LEFT JOIN` includes all students
- `COUNT()` counts borrowed books

Query 4: Most borrowed book

```
SELECT b.title, COUNT(*) AS times_borrowed
FROM Borrow br
JOIN Books b ON br.book_id = b.book_id
GROUP BY b.title
ORDER BY times_borrowed DESC
LIMIT 1;
```

Stored Procedures

Procedure 1: Issue Book

```
DELIMITER //

CREATE PROCEDURE IssueBook(
    IN p_student INT,
```

```
    IN p_book INT,
    IN p_librarian INT
)
BEGIN
    DECLARE available INT;

    SELECT available_copies INTO available
    FROM Books WHERE book_id = p_book;

    IF available > 0 THEN
        INSERT INTO Borrow(student_id, book_id, librarian_id,
issue_date)
        VALUES (p_student, p_book, p_librarian, CURDATE());

        UPDATE Books
        SET available_copies = available_copies - 1
        WHERE book_id = p_book;

    ELSE
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Book not available';
    END IF;
END //

DELIMITER ;
```

- Checks availability
- Inserts borrow record
- Decreases stock
- Throws error if unavailable

Procedure 2: Return Book

```
DELIMITER //

CREATE PROCEDURE ReturnBook(IN p_borrow_id INT)
BEGIN
    DECLARE b_id INT;

    SELECT book_id INTO b_id
    FROM Borrow WHERE borrow_id = p_borrow_id;

    UPDATE Borrow
    SET return_date = CURDATE()
```

```
    WHERE borrow_id = p_borrow_id;

    UPDATE Books
    SET available_copies = available_copies + 1
    WHERE book_id = b_id;
END //

DELIMITER ;
```

- Updates return date
- Increases available copies

Functions

Function 1: Calculate Fine

```
DELIMITER //

CREATE FUNCTION CalculateFine(issue DATE, ret DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE days INT;
    DECLARE fine INT;

    SET days = DATEDIFF(ret, issue);

    IF days > 7 THEN
        SET fine = (days - 7) * 5;
    ELSE
        SET fine = 0;
    END IF;

    RETURN fine;
END //

DELIMITER ;
```

- First 7 days → no fine
- After that → ₹5/day

Function 2: Total Borrowed Books by Student

```
DELIMITER //

CREATE FUNCTION TotalBooksBorrowed(sid INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*) INTO total
    FROM Borrow
    WHERE student_id = sid;

    RETURN total;
END //

DELIMITER ;
```

Advanced Query using Function

```
SELECT
    s.name,
    b.title,
    CalculateFine(br.issue_date, br.return_date) AS fine
FROM Borrow br
JOIN Students s ON br.student_id = s.student_id
JOIN Books b ON br.book_id = b.book_id
WHERE br.return_date IS NOT NULL;
```

Transaction Example (Concurrency Control)

```
START TRANSACTION;

CALL IssueBook(1, 101, 1);

COMMIT;
```

If error:

```
ROLLBACK;
```

Students who borrowed ALL books

```
SELECT s.name
FROM Students s
WHERE NOT EXISTS (
    SELECT b.book_id
    FROM Books b
    WHERE NOT EXISTS (
        SELECT *
        FROM Borrow br
        WHERE br.student_id = s.student_id
        AND br.book_id = b.book_id
    )
);
```

- Double `NOT EXISTS`
- Finds students who borrowed every book